1. Frozen backbone

```
--- Starting task1_frozen ---
Downloading: "https://download.pytorch.org/models/efficientnet_b0_rwightman-7f
100%|          | 20.5M/20.5M [00:00<00:00, 92.0MB/s]
Epoch [1/5] | Train Loss: 1.4462, Acc: 64.19% | Val Loss: 0.7386, Acc: 81.53%
Epoch [2/5] | Train Loss: 0.7475, Acc: 80.42% | Val Loss: 0.5335, Acc: 87.05%
Epoch [3/5] | Train Loss: 0.5793, Acc: 83.96% | Val Loss: 0.5324, Acc: 89.17%
Epoch [4/5] | Train Loss: 0.5045, Acc: 85.76% | Val Loss: 0.3707, Acc: 90.80%
Epoch [5/5] | Train Loss: 0.4554, Acc: 86.90% | Val Loss: 0.3262, Acc: 91.65%
```

Train acc: 86.9 Val acc: 91.65

2. Partially frozen backbone

```
--- Starting task3_partial_freeze ---
Epoch [1/5] | Train Loss: 0.2565, Acc: 93.24% | Val Loss: 0.0222, Acc: 99.32%
Epoch [2/5] | Train Loss: 0.0389, Acc: 98.92% | Val Loss: 0.0183, Acc: 99.64%
Epoch [3/5] | Train Loss: 0.0337, Acc: 99.07% | Val Loss: 0.0125, Acc: 99.64%
Epoch [4/5] | Train Loss: 0.0261, Acc: 99.31% | Val Loss: 0.0401, Acc: 98.82%
Epoch [5/5] | Train Loss: 0.0214, Acc: 99.42% | Val Loss: 0.0126, Acc: 99.64%
```

Train acc: 99.42 Val acc: 99.64

3. Fine tune all

```
--- Starting task2_fine_tune_all ---
Epoch [1/5] | Train Loss: 0.2538, Acc: 93.22% | Val Loss: 0.0319, Acc: 99.21%
Epoch [2/5] | Train Loss: 0.0470, Acc: 98.74% | Val Loss: 0.0082, Acc: 99.76%
Epoch [3/5] | Train Loss: 0.0246, Acc: 99.29% | Val Loss: 0.0084, Acc: 99.74%
Epoch [4/5] | Train Loss: 0.0245, Acc: 99.31% | Val Loss: 0.0127, Acc: 99.61%
Epoch [5/5] | Train Loss: 0.0206, Acc: 99.52% | Val Loss: 0.0043, Acc: 99.92%
```

Train acc:99.52 Val acc:99.92

Q1) The frozen backbone converges fastest as it only has to update weights of the final layer so there are way fewer parameters to optimize.
Q2) The fully fine tuned(3) showed the most overfitting because there are a huge number of weights that can be trained and due to the small size of the dataset the model can basically memorise specific things in the dataset instead of general traffic sign features.
Q3) The MNIST images contain very simple features like edges and strokes and the earlier layers of models like efficientnet are trained to detect these kinds of features so freezing them lets us keep this type of classification and doesn't let the model get distorted.
Q4) Full fine tuning is necessary when your dataset is extremely large and the target images are significantly different from the initial data that the model was trained on.